

SQL

SQL (Structured Query Language) es un lenguaje de programación utilizado para administrar y manipular bases de datos relacionales. Se utiliza para la gestión de datos en todo tipo de aplicaciones, desde pequeñas aplicaciones de escritorio hasta grandes sistemas empresariales.

La importancia de SQL radica en que es la forma estándar y más utilizada para trabajar con bases de datos relacionales, lo que lo hace esencial en la mayoría de los proyectos de análisis de datos, ciencia de datos, inteligencia artificial y aplicaciones de negocios. SQL permite a los usuarios consultar, actualizar y manipular datos de manera eficiente y segura, proporcionando una forma flexible y poderosa de almacenar y acceder a información crítica de manera rápida y confiable.

En el análisis de datos, SQL se utiliza para realizar consultas complejas y análisis de datos para obtener información valiosa de grandes conjuntos de datos. En la ciencia de datos, SQL se utiliza para preprocesar datos, limpiar datos, realizar consultas y unir varias fuentes de datos. En inteligencia artificial, SQL se utiliza para almacenar y acceder a datos de entrenamiento de modelos de aprendizaje automático. En las aplicaciones de negocios, SQL se utiliza para la gestión de bases de datos de clientes, facturación y seguimiento de inventario, entre otras tareas críticas.

Manipulación de tablas en SQL

Operaciones con tablas

Para **crear** una tabla en SQL Server se utiliza el comando CREATE TABLE, seguido del nombre de la tabla y la lista de columnas con sus respectivos tipos de datos y restricciones. Por ejemplo:

```
CREATE TABLE NombreTabla (  
    Columna1 TipoDato1 [Restricciones],  
    Columna2 TipoDato2 [Restricciones],  
    ...  
    ColumnaN TipoDatoN [Restricciones]  
);
```

Para **modificar** una tabla existente en SQL Server se utiliza el comando ALTER TABLE, seguido del nombre de la tabla y la instrucción de modificación. Por ejemplo:

```
ALTER TABLE NombreTabla ADD ColumnaN TipoDatoN [Restricciones];
ALTER TABLE NombreTabla DROP COLUMN Columna;
ALTER TABLE NombreTabla MODIFY Columna TipoDato [Restricciones];
```

Para eliminar una tabla en SQL Server se utiliza el comando DROP TABLE, seguido del nombre de la tabla. Por ejemplo:

```
DROP TABLE NombreTabla;
```

Para asignar **tipos de datos** a las columnas de una tabla en SQL Server se utilizan los siguientes tipos de datos:

- VARCHAR: para almacenar cadenas de caracteres de longitud variable.
- CHAR: para almacenar cadenas de caracteres de longitud fija.
- INT: para almacenar números enteros.
- FLOAT: para almacenar números decimales.
- DATE: para almacenar fechas.
- TIME: para almacenar tiempos.
- DATETIME: para almacenar fechas y tiempos.

Para establecer relaciones entre tablas en SQL Server se utilizan las siguientes instrucciones:

- PRIMARY KEY: para indicar que una columna es clave primaria de la tabla.
- FOREIGN KEY: para indicar que una columna es clave foránea que hace referencia a otra tabla.
- REFERENCES: para indicar la tabla y columna a la que hace referencia la clave foránea.

Por ejemplo:

```
CREATE TABLE Tabla1 (  
    Columna1 INT PRIMARY KEY,  
    Columna2 VARCHAR(50)  
);  
  
CREATE TABLE Tabla2 (  
    Columna3 INT PRIMARY KEY,
```

```
Columna4 VARCHAR(50),
Columna5 INT FOREIGN KEY REFERENCES Tabla1(Columna1)
);
```

Espero que esta información sea de ayuda para ti y tus estudiantes. Si tienes alguna otra pregunta, no dudes en preguntarme.

A continuación, te recomiendo modelos de tarjeta de memoria para cada uno de los casos anteriores, lo cual facilitará tu estudio, puedes usar estos modelos para estudiar el resto del contenido de este documento:

- Crear una tabla en SQL Server:

```
Frontal:
CREATE TABLE NombreTabla (
    Columna1 TipoDato1 [Restricciones],
    Columna2 TipoDato2 [Restricciones],
    ...
    ColumnaN TipoDatoN [Restricciones]
);
```

Reverso:

Ejemplo:

```
CREATE TABLE Empleados (
    IdEmpleado INT PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL,
    FechaNacimiento DATE,
    Salario FLOAT
);
```

- Modificar una tabla existente en SQL Server:

```
Frontal:
ALTER TABLE NombreTabla ADD ColumnaN TipoDatoN [Restricciones];
ALTER TABLE NombreTabla DROP COLUMN Columna;
ALTER TABLE NombreTabla MODIFY Columna TipoDato [Restricciones];
```

Reverso:

Ejemplo:

```
ALTER TABLE Empleados ADD Departamento VARCHAR(50);
ALTER TABLE Empleados DROP COLUMN Salario;
ALTER TABLE Empleados MODIFY Nombre NVARCHAR(50) NOT NULL;
```

- Eliminar una tabla en SQL Server:

Frontal:

```
DROP TABLE NombreTabla;
```

Reverso:

Ejemplo:

```
DROP TABLE Empleados;
```

- Asignar tipos de datos a las columnas de una tabla en SQL Server:

Frontal:

Tipos de datos en SQL Server:

- VARCHAR
- CHAR
- INT
- FLOAT
- DATE
- TIME
- DATETIME

Reverso:

Ejemplo:

```
CREATE TABLE Ventas (  
    IdVenta INT PRIMARY KEY,  
    FechaVenta DATE,  
    HoraVenta TIME,  
    TotalVenta FLOAT  
);
```

- Establecer relaciones entre tablas en SQL Server:

Frontal:

Instrucciones para establecer relaciones entre tablas en SQL Server:

- PRIMARY KEY
- FOREIGN KEY
- REFERENCES

Reverso:

Ejemplo:

```
CREATE TABLE Pedidos (  
    IdPedido INT PRIMARY KEY,  
    FechaPedido DATE,  
    IdCliente INT FOREIGN KEY REFERENCES Clientes(IdCliente)  
);
```

```
CREATE TABLE Clientes (  
    IdCliente INT PRIMARY KEY,  
    Nombre VARCHAR(50) NOT NULL,  
    Apellido VARCHAR(50) NOT NULL
```

```
);
```

Espero que estas tarjetas de memoria sean de ayuda para tus estudiantes. Si necesitas más información, no dudes en preguntar.

Ejercicio consolidador

```
-- Crear una tabla en SQL Server
CREATE TABLE Empleados (
    IdEmpleado INT PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL,
    FechaNacimiento DATE,
    Departamento VARCHAR(50)
);

-- Modificar una tabla existente en SQL Server
ALTER TABLE Empleados ADD Salario FLOAT;
ALTER TABLE Empleados DROP COLUMN Departamento;
ALTER TABLE Empleados ALTER COLUMN Nombre NVARCHAR(50) NOT NULL;

-- Eliminar una tabla en SQL Server
DROP TABLE Ventas;

-- Asignar tipos de datos a las columnas de una tabla en SQL Server
CREATE TABLE Pedidos (
    IdPedido INT PRIMARY KEY,
    FechaPedido DATE,
    HoraPedido TIME,
    TotalPedido FLOAT,
    IdEmpleado INT FOREIGN KEY REFERENCES Empleados(IdEmpleado)
);

-- Establecer relaciones entre tablas en SQL Server
CREATE TABLE Clientes (
    IdCliente INT PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL
);

CREATE TABLE DetallesPedido (
    IdDetalle INT PRIMARY KEY,
    IdPedido INT FOREIGN KEY REFERENCES Pedidos(IdPedido),
    IdProducto INT FOREIGN KEY REFERENCES Productos(IdProducto),
    Cantidad INT,
    Precio FLOAT
);

CREATE TABLE Productos (
    IdProducto INT PRIMARY KEY,
    NombreProducto VARCHAR(50) NOT NULL,
    PrecioProducto FLOAT
```

```
);
```

```
-- Agregar columna IdCliente a la tabla DetallesPedido
ALTER TABLE DetallesPedido
ADD IdCliente INT;

-- Agregar clave foránea a la columna IdCliente de la tabla DetallesPedido
ALTER TABLE DetallesPedido
ADD CONSTRAINT FK_DetallesPedido_Clientes
FOREIGN KEY (IdCliente)
REFERENCES Clientes(IdCliente);
```

Operaciones sobre tablas

A continuación, te menciono algunas de ellas:

- **INSERT INTO** : Permite insertar nuevos datos en una tabla. Su sintaxis es la siguiente: `INSERT INTO nombreTabla (columna1, columna2, ..., columnaN) VALUES (valor1, valor2, ..., valorN);`
- **UPDATE** : Permite actualizar los datos existentes en una tabla. Su sintaxis es la siguiente: `UPDATE nombreTabla SET columna1 = valor1, columna2 = valor2, ..., columnaN = valorN WHERE condicion;`
- **DELETE** : Permite eliminar filas de una tabla. Su sintaxis es la siguiente: `DELETE FROM nombreTabla WHERE condicion;`
- **SELECT** : Permite seleccionar datos de una o varias tablas. Su sintaxis es la siguiente: `SELECT columna1, columna2, ..., columnaN FROM nombreTabla WHERE condicion;`
- **GROUP BY** : Agrupa los datos de una tabla en función de una o varias columnas. Su sintaxis es la siguiente: `SELECT columna1, SUM(columna2) FROM nombreTabla GROUP BY columna1;`
- **HAVING** : Permite filtrar los resultados de una consulta después de haber aplicado la cláusula GROUP BY. Su sintaxis es la siguiente: `SELECT columna1, SUM(columna2) FROM nombreTabla GROUP BY columna1 HAVING SUM(columna2) > valor;`
- **ORDER BY** : Ordena los resultados de una consulta en función de una o varias columnas. Su sintaxis es la siguiente: `SELECT columna1, columna2, ..., columnaN FROM nombreTabla ORDER BY columna1 DESC, columna2 ASC;`

Estas son solo algunas de las operaciones que se pueden realizar con tablas en SQL, pero hay muchas más. Cada una de ellas tiene su sintaxis y sus particularidades, pero

todas tienen en común que permiten trabajar con los datos almacenados en una o varias tablas de una base de datos.

1. INSERT INTO:

Sintaxis: `INSERT INTO nombreTabla (columna1, columna2, ..., columnaN) VALUES (valor1, valor2, ..., valorN);`

Ejemplo de uso:

```
INSERT INTO estudiantes (nombre, edad, carrera) VALUES ('Juan', 23, 'Ingeniería en Sistemas');
```

Casos en que se puede emplear:

- Para insertar nuevos datos en una tabla.

2. UPDATE:

Sintaxis: `UPDATE nombreTabla SET columna1 = valor1, columna2 = valor2, ..., columnaN = valorN WHERE condicion;`

Ejemplo de uso:

```
UPDATE estudiantes SET edad = 24 WHERE id = 1;
```

Casos en que se puede emplear:

- Para actualizar los datos existentes en una tabla.

3. DELETE:

Sintaxis: `DELETE FROM nombreTabla WHERE condicion;`

Ejemplo de uso:

```
DELETE FROM estudiantes WHERE carrera = 'Ingeniería en Sistemas';
```

Casos en que se puede emplear:

- Para seleccionar datos de una o varias tablas.

5. GROUP BY:

Sintaxis: `SELECT columna1, SUM(columna2) FROM nombreTabla GROUP BY columna1;`

Ejemplo de uso:

```
SELECT carrera, COUNT(*) FROM estudiantes GROUP BY carrera;
```

Casos en que se puede emplear:

- Para agrupar los datos de una tabla en función de una o varias columnas.

6. HAVING:

Sintaxis: `SELECT columna1, SUM(columna2) FROM nombreTabla GROUP BY columna1 HAVING SUM(columna2) > valor;`

Ejemplo de uso:

```
SELECT carrera, AVG(edad) FROM estudiantes GROUP BY carrera HAVING AVG(edad) > 22;
```

Casos en que se puede emplear:

- Para filtrar los resultados de una consulta después de haber aplicado la cláusula GROUP BY.

7. ORDER BY:

Sintaxis: `SELECT columna1, columna2, ..., columnaN FROM nombreTabla ORDER BY columna1 DESC, columna2 ASC;`

Ejemplo de uso:

```
SELECT nombre, edad FROM estudiantes ORDER BY edad DESC;
```

Casos en que se puede emplear:

- Para ordenar los resultados de una consulta en función de una o varias columnas.

Estructuremos correctamente una consulta en SQL

La **estructura básica de una sentencia en SQL** consta de tres partes principales: la cláusula **SELECT**, la cláusula **FROM** y la cláusula **WHERE**. Dependiendo de la operación que se esté realizando, pueden agregarse otras cláusulas y partes adicionales.

A continuación, detallo cada una de las partes de una sentencia en SQL:

1. Cláusula **SELECT**: Esta cláusula especifica qué columnas se van a seleccionar en la consulta. Su sintaxis es:

```
SELECT columna1, columna2, ..., columnaN
```

En donde **columna1**, **columna2**, ..., **columnaN** son los nombres de las columnas que se desean seleccionar. Se pueden seleccionar todas las columnas de una tabla utilizando el caracter ***** en lugar de la lista de nombres de columnas.

2. Cláusula **FROM**: Esta cláusula indica la tabla o tablas en las que se va a realizar la consulta. Su sintaxis es:

```
FROM nombreTabla
```

En donde **nombreTabla** es el nombre de la tabla de la que se desea seleccionar datos. En caso de querer seleccionar datos de más de una tabla, se deben separar los nombres de las tablas con comas.

3. Cláusula **WHERE**: Esta cláusula especifica las condiciones que deben cumplir los datos que se seleccionen. Su sintaxis es:

```
WHERE condicion
```

En donde **condicion** es una expresión lógica que indica las condiciones que deben cumplir las filas de la tabla para ser seleccionadas. Por ejemplo, **edad > 18** sería una condición para seleccionar todas las filas de la tabla donde la edad sea

mayor a 18. Otras cláusulas y partes adicionales que pueden agregarse a una sentencia SQL incluyen:

- Cláusula ORDER BY: Esta cláusula permite ordenar los resultados de la consulta en base a una o varias columnas. Su sintaxis es:

```
ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC], ..., columnaN [ASC|DESC]
```

En donde **columna1**, **columna2**, ..., **columnaN** son las columnas por las que se desea ordenar los resultados, y **ASC** indica orden ascendente (de menor a mayor) y **DESC** indica orden descendente (de mayor a menor).

- Cláusula GROUP BY: Esta cláusula permite agrupar los resultados de la consulta por una o varias columnas. Su sintaxis es:

```
GROUP BY columna1, columna2, ..., columnaN
```

En donde **columna1**, **columna2**, ..., **columnaN** son las columnas por las que se desea agrupar los resultados.

- Cláusula HAVING: Esta cláusula permite filtrar los resultados de una consulta después de haber aplicado la cláusula GROUP BY. Su sintaxis es similar a la de la cláusula WHERE:

```
HAVING condicion
```

En donde **condicion** es una expresión lógica que indica las condiciones que deben cumplir los grupos de resultados.

Ejercicios sugeridos para practicar

1. Crea una tabla llamada "Clientes" con las columnas "IdCliente", "Nombre" y "Apellido". Luego, inserta 3 clientes diferentes en la tabla.
2. Crea una tabla llamada "Productos" con las columnas "IdProducto", "Nombre" y "Precio". Luego, inserta 5 productos diferentes en la tabla.
3. Crea una tabla llamada "Pedidos" con las columnas "IdPedido", "IdCliente" y "Fecha". Luego, inserta 3 pedidos diferentes en la tabla.

4. Actualiza el precio del producto con `IdProducto = 2` a 10.99.
5. Elimina el cliente con `IdCliente = 1` de la tabla "Clientes".
6. Selecciona el nombre y apellido de todos los clientes que tengan un pedido hecho.
7. Selecciona el nombre del producto y su precio de todos los productos cuyo precio sea mayor a 5.
8. Crea una tabla llamada "Empleados" con las columnas "IdEmpleado", "Nombre" y "Apellido". Luego, crea una relación entre la tabla "Pedidos" y la tabla "Empleados" de tal forma que la columna "IdEmpleado" en la tabla "Pedidos" haga referencia a la columna "IdEmpleado" en la tabla "Empleados".
9. Crea una tabla llamada "DetallesPedido" con las columnas "IdDetalle", "IdPedido", "IdProducto", "Cantidad" y "Precio". Luego, crea una relación entre la tabla "Pedidos" y la tabla "DetallesPedido" de tal forma que la columna "IdPedido" en la tabla "DetallesPedido" haga referencia a la columna "IdPedido" en la tabla "Pedidos". Crea también una relación entre la tabla "Productos" y la tabla "DetallesPedido" de tal forma que la columna "IdProducto" en la tabla "DetallesPedido" haga referencia a la columna "IdProducto" en la tabla "Productos".
10. Selecciona el nombre del cliente, el nombre del producto y la cantidad del producto comprada para todos los pedidos en la tabla "Pedidos" junto con el precio total de cada pedido ($\text{cantidad} * \text{precio}$) en la tabla "DetallesPedido". Ordena los resultados por fecha del pedido de forma descendente.

Diccionarios de datos

Crear un diccionario de datos es un proceso importante para cualquier proyecto de base de datos, ya que nos permite documentar y organizar la información que se almacenará en la base de datos. Aquí hay una serie de pasos que se pueden seguir para crear un diccionario de datos:

1. Identificar las tablas y columnas que se incluirán en el diccionario de datos.
2. Para cada tabla, identificar las siguientes características:
 - Nombre de la tabla
 - Descripción de la tabla
 - Clave primaria
 - Claves foráneas (si las hay)
 - Índices (si los hay)
3. Para cada columna de cada tabla, identificar las siguientes características:
 - Nombre de la columna

- Descripción de la columna
 - Tipo de datos de la columna
 - Tamaño de la columna (si es aplicable)
 - Restricciones (por ejemplo, NOT NULL, UNIQUE)
 - Valor por defecto (si lo hay)
4. Crear una tabla en el diccionario de datos para cada tabla en la base de datos.
Cada tabla del diccionario de datos debe incluir las características identificadas en el paso 2.
 5. Para cada columna en cada tabla del diccionario de datos, crear una fila que incluya las características identificadas en el paso 3.
 6. Asegurarse de que el diccionario de datos esté actualizado a medida que se agreguen, modifiquen o eliminen tablas y columnas de la base de datos.

Aquí hay un formato de tabla de muestra que se puede utilizar para crear un diccionario de datos:

Nombre de la tabla	Descripción de la tabla	Clave primaria	Claves foráneas	Índices	
tabla_1	Descripción de la tabla_1	columna_1	-	-	
tabla_2	Descripción de la tabla_2	columna_1, columna_2	columna_3 (tabla_3)	índice_1 (columna_1), índice_2 (columna_2)	
Nombre de la columna	Descripción de la columna	Tipo de datos	Tamaño	Restricciones	Valor por defecto
columna_1	Descripción de la columna_1	INT	-	NOT NULL, UNIQUE	-
columna_2	Descripción de la columna_2	VARCHAR	50	-	-
columna_3	Descripción de la columna_3	DATE	-	-	'01/01/2000'

En este formato, cada tabla se representa como una fila en la primera tabla, y cada columna se representa como una fila en la segunda tabla. Las columnas "Claves foráneas" e "Índices" en la primera tabla pueden tener varios valores separados por comas si hay varias claves foráneas o índices en una tabla.

Anexos

Tabla de resumen 1

Sentencia	Sintaxis	Ejemplo de Uso	Casos de Uso
CREATE TABLE	CREATE TABLE nombreTabla (columna1 tipoDato1, columna2 tipoDato2, ..., columnaN tipoDatoN);	CREATE TABLE Empleados (IdEmpleado INT PRIMARY KEY, Nombre VARCHAR(50) NOT NULL, Apellido VARCHAR(50) NOT NULL);	Crear una nueva tabla en la base de datos.
ALTER TABLE	ALTER TABLE nombreTabla ADD nombreColumna tipoDato; `ALTER TABLE nombreTabla DROP COLUMN nombreColumna;  `ALTER TABLE nombreTabla MODIFY nombreColumna nuevoTipoDato;	ALTER TABLE Empleados ADD Salario FLOAT; `ALTER TABLE Empleados DROP COLUMN Departamento;  `ALTER TABLE Empleados MODIFY Nombre NVARCHAR(50) NOT NULL;	Modificar una tabla existente en la base de datos.
DROP TABLE	DROP TABLE nombreTabla;	DROP TABLE Ventas;	Eliminar una tabla existente en la base de datos.
CREATE FOREIGN KEY	CREATE TABLE nombreTabla1 (columna1 tipoDato1, columna2 tipoDato2, ..., columnaN tipoDatoN, FOREIGN KEY	CREATE TABLE Pedidos (IdPedido INT PRIMARY KEY, FechaPedido DATE,	Establecer una relación entre dos

Sentencia	Sintaxis	Ejemplo de Uso	Casos de Uso
	(nombreColumna) REFERENCES nombreTabla2(nombreColumna));	HoraPedido TIME, TotalPedido FLOAT, IdEmpleado INT FOREIGN KEY REFERENCES Empleados(IdEmpleado));	tablas en la base de datos.
CREATE PRIMARY KEY	CREATE TABLE nombreTabla (columna1 tipoDato1 PRIMARY KEY, columna2 tipoDato2, ..., columnaN tipoDatoN);	CREATE TABLE Empleados (IdEmpleado INT PRIMARY KEY, Nombre VARCHAR(50) NOT NULL, Apellido VARCHAR(50) NOT NULL);	Especificar una clave primaria en una tabla.
CREATE INDEX	CREATE INDEX nombreIndice ON nombreTabla (nombreColumna);	CREATE INDEX idx_Nombre ON Empleados (Nombre);	Agilizar el rendimiento de consultas SQL.

Tabla de Resumen 2.

Operación	Sintaxis	Ejemplo de uso	Casos de uso
INSERT INTO	INSERT INTO nombreTabla (columna1, columna2, ..., columnaN) VALUES (valor1, valor2, ..., valorN);	INSERT INTO estudiantes (nombre, edad, carrera) VALUES ('Juan', 23, 'Ingeniería en Sistemas');	Insertar nuevos datos en una tabla.
UPDATE	UPDATE nombreTabla SET columna1 = valor1, columna2 = valor2, ...,	UPDATE estudiantes SET edad = 24 WHERE id = 1;	Actualizar los datos existentes en una tabla.

Operación	Sintaxis	Ejemplo de uso	Casos de uso
	columnaN = valorN WHERE condicion;		
DELETE	DELETE FROM nombreTabla WHERE condicion;	DELETE FROM estudiantes WHERE id = 1;	Eliminar filas de una tabla.
SELECT	SELECT columna1, columna2, ..., columnaN FROM nombreTabla WHERE condicion;	SELECT nombre, edad FROM estudiantes WHERE carrera = 'Ingeniería en Sistemas';	Seleccionar datos de una o varias tablas.
GROUP BY	SELECT columna1, SUM(columna2) FROM nombreTabla GROUP BY columna1;	SELECT carrera, COUNT(*) FROM estudiantes GROUP BY carrera;	Agrupar los datos de una tabla en función de una o varias columnas.
HAVING	SELECT columna1, SUM(columna2) FROM nombreTabla GROUP BY columna1 HAVING SUM(columna2) > valor;	SELECT carrera, AVG(edad) FROM estudiantes GROUP BY carrera HAVING AVG(edad) > 22;	Filtrar los resultados de una consulta después de haber aplicado la cláusula GROUP BY.
ORDER BY	SELECT columna1, columna2, ..., columnaN FROM nombreTabla ORDER BY columna1 DESC, columna2 ASC;	SELECT nombre, edad FROM estudiantes ORDER BY edad DESC;	Ordenar los resultados de una consulta en función de una o varias columnas.

Soluciones a los ejercicios propuestos

```
-- Crear la tabla "Clientes"
CREATE TABLE Clientes (
    IdCliente INT PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL
);
```

```
-- Insertar 3 clientes diferentes en la tabla
INSERT INTO Clientes (IdCliente, Nombre, Apellido) VALUES (1, 'Juan', 'Pérez');
INSERT INTO Clientes (IdCliente, Nombre, Apellido) VALUES (2, 'María', 'González');
INSERT INTO Clientes (IdCliente, Nombre, Apellido) VALUES (3, 'Pedro', 'Martínez');
```

```
-- Crear la tabla "Productos"
CREATE TABLE Productos (
    IdProducto INT PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Precio FLOAT
);

-- Insertar 5 productos diferentes en la tabla
INSERT INTO Productos (IdProducto, Nombre, Precio) VALUES (1, 'Camisa', 15.99);
INSERT INTO Productos (IdProducto, Nombre, Precio) VALUES (2, 'Pantalón', 25.99);
INSERT INTO Productos (IdProducto, Nombre, Precio) VALUES (3, 'Zapatos', 30.99);
INSERT INTO Productos (IdProducto, Nombre, Precio) VALUES (4, 'Chaqueta', 50.99);
INSERT INTO Productos (IdProducto, Nombre, Precio) VALUES (5, 'Sombrero', 10.99);
```

```
-- Crear la tabla "Pedidos"
CREATE TABLE Pedidos (
    IdPedido INT PRIMARY KEY,
    IdCliente INT,
    Fecha DATE
);

-- Insertar 3 pedidos diferentes en la tabla
INSERT INTO Pedidos (IdPedido, IdCliente, Fecha) VALUES (1, 1, '2023-04-30');
INSERT INTO Pedidos (IdPedido, IdCliente, Fecha) VALUES (2, 2, '2023-05-01');
INSERT INTO Pedidos (IdPedido, IdCliente, Fecha) VALUES (3, 3, '2023-05-02');
```

```
UPDATE Productos
SET Precio = 10.99
WHERE IdProducto = 2;
```

```
DELETE FROM Clientes
WHERE IdCliente = 1;
```

```
SELECT c.Nombre, c.Apellido
FROM Clientes c
INNER JOIN Pedidos p ON c.IdCliente = p.IdCliente;
```



```
SELECT Nombre, Precio
FROM Productos
WHERE Precio > 5;
```

```
CREATE TABLE Empleados (
    IdEmpleado INT PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL
);

ALTER TABLE Pedidos
ADD FOREIGN KEY (IdEmpleado) REFERENCES Empleados(IdEmpleado);
```

```
CREATE TABLE DetallesPedido (
    IdDetalle INT PRIMARY KEY,
    IdPedido INT FOREIGN KEY REFERENCES Pedidos(IdPedido),
    IdProducto INT FOREIGN KEY REFERENCES Productos(IdProducto),
    Cantidad INT,
    Precio FLOAT
);

ALTER TABLE DetallesPedido
ADD FOREIGN KEY (IdPedido) REFERENCES Pedidos(IdPedido);

ALTER TABLE DetallesPedido
ADD FOREIGN KEY (IdProducto) REFERENCES Productos(IdProducto);
```

```
SELECT Clientes.Nombre, Productos.NombreProducto, DetallesPedido.Cantidad,
DetallesPedido.Precio * DetallesPedido.Cantidad AS 'Precio Total'
FROM Pedidos
INNER JOIN Clientes ON Pedidos.IdCliente = Clientes.IdCliente
INNER JOIN DetallesPedido ON Pedidos.IdPedido = DetallesPedido.IdPedido
INNER JOIN Productos ON DetallesPedido.IdProducto = Productos.IdProducto
ORDER BY Pedidos.FechaPedido DESC;
```